

Activity 5

Week 6

6733172621 Patthadon Phengpinij

Collaborators. Claude (for L^AT_EX styling and grammar checking)

Activity: DevOps and Infrastructure as Code

Instructions

1. This assignment is due Wednesday September 16, 2026 no later than 11:00 pm on mycourseville (pdf only).
2. Use terraform to run `todo` container version 3.0 and `redis` container.
3. Prepare a PDF report containing the following information:
 - (a) Content of your terraform file.
 - (b) Screenshot of running terraform in each step.

YOU MUST ALSO USING GRADER (URL is in MCV's assignment)

Environment. Everything runs on a local Docker Engine 29.7.2 (Ubuntu on WSL2), driven by **Terraform v1.13.3** through the **kreuzwerker/docker** provider v3.9.0 — the community provider that talks to the Docker Engine API directly, so no `docker run` or `docker compose` is involved: the containers are created, tracked in `terraform.tfstate`, and torn down entirely as Terraform-managed resources.

A note on the third container. Version 3.0 of `todo` (`natawut/todo-service:release-3`) is the release that gained notifications: every `POST /` calls `set_notification()`, which fires a request at `NOTIFICATION_HOST:NOTIFICATION_PORT` (default `localhost:9000`) and never looks at the reply. The reply is ignored, but a *refused* connection is not: requests raises `ConnectionError` out of the request handler, so with `todo` and `redis` alone every create answers `500 Internal Server Error`, and the grader's `POST` check fails. This activity has no notification service to point it at, so a plain `nginx` stands in as that endpoint: it answers `405` to the notification `POST`, `todo` discards the reply, and the create succeeds. It is one `docker_container` resource and one variable (`var.notification_image`); pointing that variable at the real notification service is the only change needed to make it real.

Problem 1. Content of terraform file.

Solution

The whole configuration lives in one file, `main.tf`, and declares seven resources: the `todo-net` bridge network, three images, and three containers. Only `todo` is published to the host (`8000:8000`); `redis` and the notification stub stay inside `todo-net`, where Docker's embedded DNS resolves them by container name — which is what `REDIS_HOST=redis` relies on. The container is named `todo-service`, the name the rest of the course's stack (and the grader) expects. Declaring the images as `docker_image` resources makes the pull part of the plan instead of a side effect of the run, and `keep_locally = true` leaves the pulled images in the local cache when the rest is destroyed.

```

1  # Activity 5 - Infrastructure as Code
2  # Run the todo service (version 3.0) and its redis database with Terraform.
3
4  terraform {
5      required_version = ">= 1.5"
6
7      required_providers {
8          docker = {
9              source = "kreuzwerker/docker"
10             version = "~> 3.0"
11         }
12     }
13 }
14
15 # Talks to the local Docker daemon. The host is left at the provider default so
16 # it follows DOCKER_HOST / unix:///var/run/docker.sock, which keeps the file
17 # portable between Docker Desktop and a plain dockerd.
18 provider "docker" {}
19
20 variable "todo_image" {
21     description = "Docker image of the todo service (version 3.0).\"
22     type        = string
23     default     = "natawut/todo-service:release-3"
24 }
25
26 variable "redis_image" {
27     description = "Docker image of the redis database.\"
28     type        = string
29     default     = "redis:7-alpine"
30 }
31
32 # todo 3.0 POSTs a notification to NOTIFICATION_HOST:NOTIFICATION_PORT on every
33 # created todo and never checks the response, but a refused connection raises
34 # out of the request handler and turns POST / into a 500. This activity has no
35 # notification service to point it at, so a plain nginx stands in as the
36 # endpoint: it answers 405 to the notification POST, todo discards the response,
37 # and creating a todo succeeds. Swap this for the real notification service and
38 # nothing else in this file has to change.
```

```
39 variable "notification_image" {
40   description = "Docker image standing in for the notification endpoint."
41   type        = string
42   default     = "nginx:alpine"
43 }
44
45 variable "todo_port" {
46   description = "Host port the todo service is published on."
47   type        = number
48   default     = 8000
49 }
50
51 # -----
52 # Network
53 # -----
54
55 # A user-defined bridge network gives both containers Docker's embedded DNS, so
56 # todo can reach redis by container name (REDIS_HOST=redis below).
57 resource "docker_network" "todo_net" {
58   name     = "todo-net"
59   driver   = "bridge"
60 }
61
62 # -----
63 # Images
64 # -----
65
66 resource "docker_image" "redis" {
67   name          = var.redis_image
68   keep_locally = true
69 }
70
71 resource "docker_image" "todo" {
72   name          = var.todo_image
73   keep_locally = true
74 }
75
76 resource "docker_image" "notification" {
77   name          = var.notification_image
78   keep_locally = true
79 }
80
81 # -----
82 # Containers
83 # -----
84
85 # Not published to the host: only the todo service needs to reach it, and it
86 # does so over todo-net.
87 resource "docker_container" "redis" {
88   name     = "redis"
89   image    = docker_image.redis.image_id
90   restart  = "unless-stopped"
91
92   networks_advanced {
93     name = docker_network.todo_net.name
94   }
95 }
96
97 # Answers the notification POST described by var.notification_image. Like redis,
98 # it is internal to todo-net only.
99 resource "docker_container" "notification" {
100   name     = "notification-service"
101   image    = docker_image.notification.image_id
102   restart  = "unless-stopped"
103
104   networks_advanced {
```

```

105     name = docker_network.todo_net.name
106   }
107 }
108
109 resource "docker_container" "todo" {
110   name     = "todo-service"
111   image    = docker_image.todo.image_id
112   restart  = "unless-stopped"
113
114   env = [
115     "REDIS_HOST=${docker_container.redis.name}",
116     "REDIS_PORT=6379",
117     "NOTIFICATION_HOST=${docker_container.notification.name}",
118     "NOTIFICATION_PORT=80",
119   ]
120
121   ports {
122     internal = 8000
123     external = var.todo_port
124   }
125
126   networks_advanced {
127     name = docker_network.todo_net.name
128   }
129
130   # Terraform already infers these from the REDIS_HOST / NOTIFICATION_HOST
131   # references above; stated explicitly so the ordering survives any edit to
132   # those lines.
133   depends_on = [
134     docker_container.redis,
135     docker_container.notification,
136   ]
137 }
138
139 # -----
140 # Outputs
141 # -----
142
143 output "todo_url" {
144   description = "Where the todo service is reachable from the host."
145   value       = "http://localhost:${var.todo_port}"
146 }
147
148 output "container_names" {
149   description = "Containers created by this configuration."
150   value = [
151     docker_container.todo.name,
152     docker_container.redis.name,
153     docker_container.notification.name,
154   ]
155 }
156

```

Problem 2. Screenshot of running terraform in each step.

Solution

Step 1 — terraform init. Downloads the kreuzwerker/docker plugin into .terraform/ and records the exact version in .terraform.lock.hcl.

```

1 $ terraform init
2
3 Initializing the backend...
4 Initializing provider plugins...
5 - Finding kreuzwerker/docker versions matching "~> 3.0"...

```

```

6 - Installing kreuzwerker/docker v3.9.0...
7 - Installed kreuzwerker/docker v3.9.0 (self-signed, key ID 0DCE698927DAF8EC)
8 Partner and community providers are signed by their developers.
9 [...]
10 Terraform has created a lock file .terraform.lock.hcl to record the provider
11 selections it made above. Include this file in your version control repository
12 so that Terraform can guarantee to make the same selections by default when
13 you run "terraform init" in the future.
14
15 Terraform has been successfully initialized!
16

```

Step 2 — terraform plan. Seven resources to add, nothing to change or destroy. Only the `docker_container.todo` block is reproduced here — it is the one carrying the environment and the published port; the other six blocks and the attributes that are only known after apply are elided.

```

1 $ terraform plan
2
3 Terraform used the selected providers to generate the following execution
4 plan. Resource actions are indicated with the following symbols:
5   + create
6
7 Terraform will perform the following actions:
8
9   # docker_container.todo will be created
10  + resource "docker_container" "todo" {
11    + env = [
12      + "NOTIFICATION_HOST=notification-service",
13      + "NOTIFICATION_PORT=80",
14      + "REDIS_HOST=redis",
15      + "REDIS_PORT=6379",
16    ]
17    + name = "todo-service"
18    + network_mode = "bridge"
19    + restart = "unless-stopped"
20    [... remaining attributes known after apply ...]
21
22    + networks_advanced {
23      + aliases = []
24      + name = "todo-net"
25    }
26
27    + ports {
28      + external = 8000
29      + internal = 8000
30      + ip = "0.0.0.0"
31      + protocol = "tcp"
32    }
33  }
34
35  [... docker_container.redis, docker_container.notification, docker_image.todo,
36   docker_image.redis, docker_image.notification, docker_network.todo_net ...]
37
38 Plan: 7 to add, 0 to change, 0 to destroy.
39
40 Changes to Outputs:
41  + container_names = [
42    + "todo-service",
43    + "redis",
44    + "notification-service",
45  ]
46  + todo_url = "http://localhost:8000"
47

```

Step 3 — terraform apply. Terraform reprints the plan and then creates the resources, respecting the dependency order it worked out: the images and the network first, then `redis` and the notification stub, then `todo-service`, which depends on both. The pulls finish in a second because the images were already in the local cache from an earlier run — `keep_locally = true` is what keeps

```

them there across a destroy.
1  $ terraform apply -auto-approve
2
3  [... same plan as above ...]
4
5  docker_image.notification: Creating...
6  docker_network.todo_net: Creating...
7  docker_image.redis: Creating...
8  docker_image.todo: Creating...
9  docker_image.todo: Creation complete after 1s [id=sha256:8564bc13a323...natawut/todo-service:
    release-3]
10 docker_image.redis: Creation complete after 1s [id=sha256:ff02b58f971e...redis:7-alpine]
11 docker_image.notification: Creation complete after 1s [id=sha256:72ba65eb42c1...nginx:alpine]
12 docker_network.todo_net: Creation complete after 2s [id=450a852a4874...]
13 docker_container.redis: Creating...
14 docker_container.notification: Creating...
15 docker_container.notification: Creation complete after 0s [id=fb81a2187631...]
16 docker_container.redis: Creation complete after 0s [id=3b64528cd7d9...]
17 docker_container.todo: Creating...
18 docker_container.todo: Creation complete after 1s [id=1e914733dee6...]
19
20 Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
21
22 Outputs:
23
24 container_names = [
25   "todo-service",
26   "redis",
27   "notification-service",
28 ]
29 todo_url = "http://localhost:8000"
30

```

Step 4 — verifying what Terraform built. All three containers are up, with port 8000 the only one published to the host, and the service does a full round trip: the POST returns the id of the new todo and the following GET reads it back out of redis.

```

1  $ docker ps --filter name=todo-service --filter name=redis --filter name=notification-service \
2    --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
3  NAMES                IMAGE                STATUS              PORTS
4  todo-service         8564bc13a323        Up 8 seconds       0.0.0.0:8000->8000/tcp
5  notification-service  72ba65eb42c1        Up 9 seconds       80/tcp
6  redis                ff02b58f971e        Up 9 seconds       6379/tcp
7
8  $ curl -X POST -d '{"title":"todo-1","detail":"the first todo","duedate":"2022-10-24
    11:00:00","tags":[],"completed":false}' http://localhost:8000
9  {"id":0}
10
11 $ curl http://localhost:8000
12 [{"id":0,"title":"todo-1","detail":"the first todo","completed":false,"duedate":"2022-10-24T11
    :00:00","tags":[]}]
13

```

Problem 3. Screenshot of the results after running the GRADER.

Solution

The grader is pointed at main.tf (its default), checks the Terraform side — file present, CLI installed, working directory initialized, plan generated — and then the running stack: both named containers, the service answering on `http://localhost:8000`, and a POST whose result comes back on the following GET. Every check passes — Result: true — after which it prompts for the Student ID and full name to submit the result. (In the transcript below the per-line timestamps are trimmed and the status glyphs the grader prints are written as OK.)

```
nodahttap@LAPTOP-UP5JACSH:~/study/cedt-3-1/CEDT-2110506-SDS/activity-05/code$ chmod +x activity5-CEDT_linux_amd64
nodahttap@LAPTOP-UP5JACSH:~/study/cedt-3-1/CEDT-2110506-SDS/activity-05/code$ ./activity5-CEDT_linux_amd64
2026/09/16 17:55:41 Current Timestamp: 2026-09-16T17:55:41+07:00
👉 Enter [REQUIRED] Terraform file path (.tf) (leave blank for 'main.tf'):
2026/09/16 17:55:44 ✅ / Terraform file path is exist.
2026/09/16 17:55:44 ✅ / Terraform is installed.
2026/09/16 17:55:46 ✅ / Terraform is initialized.
2026/09/16 17:55:48 ✅ / Terraform plan is generated.
2026/09/16 17:55:49 ✅ / Container todo-service exists.
2026/09/16 17:55:49 ✅ / Container redis exists.
2026/09/16 17:55:49 ✅ / All specified containers are running.
2026/09/16 17:55:49 ✅ / URL http://localhost:8000 returns 200.
2026/09/16 17:55:49 ✅ / Todo is up and running at http://localhost:8000
2026/09/16 17:55:49 ✅ / POST request successful
2026/09/16 17:55:49 ✅ / POST request to http://localhost:8000 was successful.
2026/09/16 17:55:49 ✅ / GET request shows result from previous POST request to http://localhost:8000.
2026/09/16 17:55:49 Result: true
2026/09/16 17:55:49 🎉 Looks good! Please enter your StudentID and Full name below
👉 StudentID: 6733172621
👉 Full Name (TH): นันทกร นันทกร
2026/09/16 17:56:01 🎉 Successfully submitted. Your lucky number is 21783833516273154
2026/09/16 17:56:01 🎉🎉🎉 Congratulations! You have completed the activity 🎉🎉🎉
2026/09/16 17:56:01 ⚠️ Don't forget! you still need to submit your assignment via MyCourseVille ⚠️
```

Figure 1: activity5-CEDT grader — all checks pass, Result: true.